

Claude Code 斜杠命令全图鉴

把每条斜杠命令按用途归类，用生活化的类比讲清楚；页面内可即时搜索和筛选。

[← 返回 AI 栏目](#)

命令行 · 斜杠命令 · 小白友好版

91

条命令

11

个分类

200字

每条白话讲解

2026.06

数据时点

📌 什么是"斜杠命令"?

Claude Code 是一个跑在**命令行**（就是程序员敲字干活的黑框框）里的 AI 助手。平时你直接打字跟它聊就行；但有些操作不必费口舌解释，敲一个以斜杠 `/` 开头的暗号就能一键触发——好比微信里输入 `/` 调出快捷功能，或对智能音箱喊一句固定口令。比如 `/clear` 清空重来、`/model` 换个 AI 大脑。下面把这些暗号按用途分好类，逐条讲清。



14

会话与上下文



3

模型与推理



12

项目设置与扩展



8

代码审查与验证



10

并行与后台



11

远程与跨设备



10

界面与配置



7

账户与用量



7

诊断与帮助



7

体验与杂项



2

云服务商配置



会话与上下文 14 / 14

管理跟 AI 的对话本身：清空、压缩、回退、续聊、分叉。

/clear `/reset` `/new`

— 换张干净桌面

你跟 Claude 聊久了，桌面上堆满了之前的草稿纸，它每次回答都要先翻一遍这堆纸，又慢又容易被旧话题带偏。/clear 就像把桌面一扫而空，换一张干净的桌子重新开聊——但抽屉里关于这个项目的笔记还在，不会丢。而且扫掉的旧草稿也没真扔，进了回收站，想找回随时用 /resume 翻出来。一句话：聊岔了、想换个全新话题，就用它重新开局，干净利落。

何时用 想彻底换个话题、重新开聊时。

↔ **辨析** 和 /compact 比：clear 清空翻新页（旧的可 /resume 找回）；compact 是压缩保留、继续聊。

/compact

— 把聊天记录压成笔记

聊到一半发现 Claude 越来越卡、越来越健忘？因为它脑子里能装的话有限，聊得太多就满了。/compact 就像把你俩这一大段对话浓缩成一页会议纪要——啰嗦的来回都删掉，只留重点，腾出脑容量继续聊，而且前因后果它还记得。你还能特别交代一句“重点把那段需求给我留住”，它就会在压缩时优先保住你在意的内容。一句话：感觉它变迟钝、快塞满了，又不想从头开始，就用它瘦个身。

何时用 对话太长变卡、但又不想清空时。

↔ **辨析** 和 /clear 比：不清空，把长对话压成纪要、前因后果都留着续聊。

/context 参数 all

— 看脑容量用了多少

Claude 的“记忆脑容量”是有上限的，但你平时看不见到底用了多少、是不是快满了。/context 就像给你的书桌拍一张彩色俯视图：哪块堆了文件、哪块还空着，一格一格用颜色标得清清楚楚，还会顺手告诉你哪里可以收拾收拾、怎么腾地方。加上 all 参数能看得更全。一句话：想知道“它脑子还剩多少空间、要不要清理”，瞄一眼这张图就心里有数了。

何时用 想知道上下文用量、要不要清理时。

/rewind /checkpoint /undo

— 时光倒流到存档点

改着改着发现越改越糟，想回到没乱动之前？/rewind 就像打游戏读档——它一路上悄悄存了好几个进度点，你可以把对话、或者它改过的代码、或者两样一起，倒回到之前任意一个干净的存档。就跟整理房间整乱了、按一下“恢复到昨天的样子”一样省心，不用手动一件件搬回去。一句话：搞砸了别慌，直接倒带回到还没出错的那一刻。

何时用 改坏了、想退回之前某个状态时。

↔ **辨析** 和 /clear 比：不是清空，而是退回之前的存档点，还能连代码一起回退。

/resume /continue

— 翻回之前的聊天

昨天聊到一半的事，今天想接着说，但新开的对话它一脸茫然？/resume 就像微信里翻聊天记录——之前每一次对话都好好存着，你报个名字或编号，就能精准翻到那一次接着聊；记不清是哪次也没关系，它会列个清单让你挑。从哪儿断的就从哪儿续，前面说过的它全记得。一句话：想接着之前某次对话往下干，用它一键回到现场。

何时用 想继续之前某次对话时。

↔ **辨析** 和 /clear 比：clear 是开新页；resume 是把【以前某一次】对话整段调回来接着聊。

/branch

— 聊天走个岔路口

聊到一个点上，你突然想“要不换个思路试试”，又怕一试把现在聊好的搞乱？/branch 就像走到岔路口，原来那条路给你原封不动留着，你另开一条小路去探探另一个方向。试得好就用，试砸了拍拍屁股走人，回头那条主路还在原地等你，半点没动。就像写方案时“另存一份再改”，怎么折腾都不影响原稿。一句话：想试别的走法又怕弄乱现状，就从这儿分个叉。

何时用 想另起一条思路试试、又要保留原对话时。

↔ **辨析** 和 /fork 比：是【你本人】切到新线、原对话冻结保留；fork 是派分身、你留在原地。

/fork

— 派个分身去干活

手头一件大活还没忙完，又冒出一件杂事得有人盯？/fork 就像派出一个跟你一模一样、记得所有来龙去脉的助理分身，让它在后台默默把你交代的那件事办了，你自己腾出手继续干正事，互不耽误。好比你在厨房炒着菜，喊个帮手去隔壁把碗洗了，他全程知道今天在忙啥、不用你重新交代。一句话：有件事想甩出去后台跑，又懒得重讲背景，派个分身最省事。

何时用 想让一个分身在后台替你办另一件事时。

↔ **辨析** 和 /branch 比：【你留在原线】，派个分身去后台办别的事；也别和 /background（把整个会话丢后台）混。

/export

— 把聊天存成文件

这次聊出来的好东西，想存下来、发给同事或者归档，总不能一句句往外抄吧？/export 就像把整段聊天记录“打印”成一份纯文字稿，要么直接存成文件，要么塞进剪贴板随手一贴就走。跟把微信聊天导出来留底一个意思，干干净净一整份，想存想发都方便。一句话：想把整段对话完整留底或转给别人，一键导出搞定。

何时用 想把整段对话存档或分享出去时。

/copy 参数 N

— 复制它刚说的话

它刚回的那段正好要用，手动选中再复制，长一点还老选不全？/copy 就像一个“一键复制最后那段话”的按钮，它最新的回复“啪”地就进了剪贴板，找个地方一贴就行。要是你想要的不是最后一条、而是再往前数第几条，加个数字 N 告诉它就好，它精准帮你拎出来那一条。一句话：想把它说的话拿去别处用，别手动选了，喊一声它替你复制。

何时用 想把它某条回复复制走时。

/recap

— 一句话回顾此次

聊着聊着自己都绕晕了，“我们刚到底在搞啥来着”？/recap 就像随时找它要一句话总结——它立马把这场对话到目前为止干了啥、聊到哪了，用一句话给你拎清楚，让你重新对上焦。好比开长会开蒙了，喊主持人“咱现在说到第几条了”，他一句话给你拉回神。一句话：自己聊岔神、想快速理清现在的进度，问它要个一句话回顾。

何时用 想快速理清当前会话进展时。

/rename

— 给这次聊天起名

对话攒多了，回头想找哪次全靠瞎猜，名字一个比一个看不懂？/rename 就像给文件夹改个一眼能认的名字——给当前这场对话起个你自己看得懂的标题，它就显示在输入框那儿，以后翻找一眼就锁定。跟手机里把“新建文件夹”改成“出差报销”一个道理，找东西不再大海捞针。一句话：想让这次对话以后好认好找，先给它起个明白名字。

何时用 想给当前对话起个好认的名字时。

/btw

— 随口问句不留痕

正聊着正事，突然想随口问个不相干的小问题，又怕把这事掺进来搅乱正题？/btw 就像聊天时凑过去小声问一句“对了顺便问下”——它给你答完，但这句插话不会被记进正式对话记录里，正经话题该咋样还咋样，干干净净。好比开会时跟旁边人耳语两句，不会被记进会议纪要。一句话：想临时插问个小事又不想污染当前对话，用它悄悄问。

何时用 想临时问个不进历史的小问题时。

↔ **辨析** 和 /branch、/fork 比：谁都不走开，只就地问一句、且【不进对话历史】。

/goal

— 定个目标让它死磕

有些活儿得反复折腾好几轮才成，你又懒得一遍遍催“接着干、再来”？/goal 就像给它下一道军令状——把“啥样才算干成”说清楚，它就一头扎进去自己连轴干，直到真正达成目标才停，中途不用你一直盯着催。好比交代装修队“这屋子收拾干净我才验收”，人家自己干完整套才喊你。不想要了，一个 /clear 就把这目标撤掉。一句话：有件需要它持续死磕的事，定个目标让它自己跑到底。

何时用 想让它锁定一个目标持续干到完成时。

/exit /quit

— 关门走人

忙完了想干净利落地关掉这个工具，不知道该敲啥？/exit 就是那个“关门走人”的开关——一声令下，直接退出命令行，跟下班关电脑、出门带上门一样自然。没有花活，就是稳稳当当地结束这一程。一句话：活儿干完了想退出，喊它一声就走。

何时用 想退出 Claude Code 命令行时。



模型与推理 3 / 3

决定用哪个 AI 大脑、让它想多深、跑多快。

/model

— 换个 AI 大脑

嫌 AI 反应慢或者觉得它不够聪明？这条命令帮你换个“大脑”。就像点外卖时选餐厅：有的快有的精，你按今天的需求挑。同一个助手可以套不同型号的脑子——要快就选轻快的，要它把难题想透就选强力的。挑好之后还会自动记住，下次打开默认就是它，不用每次重选。另外按方向键还能调它“想多深”，边挑边试到顺手为止。换完立刻生效，不满意随时再换。

何时用 觉得 AI 不够聪明、太慢，或想换个常用型号时。

/effort 参数 low~ultracode / auto

— 调 AI 动脑强度

同一个问题，你想让 AI 随手答还是憋大招深思？这条命令就是它的“用功档位”。好比写作业：抄答案是 low，认真演算是 high，反复检查死磕到底是 max。简单活儿开低档，又快又省；烧脑的难题开高档，它会多想几步、答得更稳。参数从 low 一路到 ultracode 越来越拼命，填 auto 就是放手让它自己看着办、恢复默认。本质是在“快”和“想得透”之间帮你拨一个平衡点，不用自己纠结。

何时用 活儿太简单想提速，或难题需要 AI 深想时。

/fast 参数 on/off

— 开顶配快车道

想要又快又不掉水准？这条命令给你开“快车道”。平常有些工具一加速就偷偷换了个低配引擎，变快但变笨。这条不一样：它让最强的那个大脑（Opus）自己跑得更快，而不是把它换成小号的——好比同一位大厨手脚麻利地出菜，菜还是那道好菜，不是换学徒来糊弄你。on 打开提速，off 关掉回常速。要的就是快得理直气壮，不用拿质量换速度。

何时用 想提速又不想 AI 变笨、降级时。



项目设置与扩展 12 / 12

教 AI 认识你的项目、接外部工具、装插件、立规矩。

/init

— 给项目建一份说明书

烦恼解决：每次让 AI 帮你改代码，它都像第一天上班的新员工，不知道你这个项目长啥样、用什么规矩。/init 就是给 AI 发一本《员工手册》。它会自己把整个项目翻一遍，写成一份叫 CLAUDE.md 的说明书，记下项目用啥技术、文件怎么放、有什么习惯。以后 AI 干活前先看这本手册，就不用你每次从头交代了。就像新保姆进门，你先递一张纸条写清楚『厨房在左边、孩子不吃辣、垃圾周二倒』，她照着做，省得你天天重复。新项目第一次用 AI，先跑这个。

何时用 刚开始用 AI 接手一个项目时，先建说明书。

/memory

— 管理 AI 的记忆本

烦恼解决：你跟 AI 说过的偏好（比如『缩进用两个空格』『别加多余注释』），它换个话题就忘，每次都得重说。/memory 就是打开 AI 的那本记忆本（CLAUDE.md 文件），让你直接翻看、添加、修改它记住的东西，还能开关『自动记忆』功能——也就是要不要让它边干边自己记笔记。就像家里冰箱门上贴的便利贴：谁过敏、谁忌口、买菜清单，全写那儿，全家照着办。你随时能撕一张、补一张。想让 AI 长期记住某个规矩，就写进这里。

何时用 想查看或修改 AI 长期记住的偏好和规矩时。

/agents

— 管理专项小助手

烦恼解决：一个全能助手什么都干，但术业有专攻——查 bug 的、写测试的、审代码的，各自专心一件事会更靠谱。/agents 就是让你组建并管理一队『专项小助手』

（子智能体），每个只负责一个领域，需要时主助手把活派给对应的人。就像家里装修，你不会让一个师傅又管水电又管刷墙又管贴砖，而是叫水电工、油漆工、瓦工各干各的，活更精细。在这里你能新建、调整、删除这些小助手，配置他们各自的专长。

想让某类任务有个固定的专家来接时用。

何时用 想配置专门处理某类任务的小助手时。

/mcp

— 给 AI 接上外部工具

烦恼解决：AI 本身只会读写你电脑里的文件，但你想让它帮你查数据库、调外部服务、接公司内部系统，它够不着。MCP 就是一根标准化的『万能转接头』，让 AI 能安全接上这些外部工具和数据。/mcp 就是管理这些接头：连了哪些、要不要授权、断开哪个。就像给手机配各种数据线和插座——单有手机没用，接上充电器、接上电视、接上音箱，它才能干更多事。这里就是你的插线板控制面板。想让 AI 用上某个外部服务，先在这里连接并授权。

何时用 想让 AI 接入外部工具或数据源时。

/hooks

— 查看自动触发的脚本

烦恼解决：有些动作你希望每次都自动做，不想手动盯着——比如 AI 每改完文件就自动格式化、每次提交前自动检查。钩子（hook）就是预先挂好的『自动机关』：某个动作一发生，它就自动触发一段脚本去干活。/hooks 让你查看当前都挂了哪些这种机关，分别在什么时候触发。就像家里的感应灯，人一进门灯自动亮，你不用去摸开关——机关是提前装好的，到点就自己动。想搞清楚 AI 在背后偷偷自动做了哪些事，来这里看。

何时用 想查看有哪些动作会自动触发脚本时。

/permissions /allowed-tools

— 设 AI 的行动权限

烦恼解决：你既想让 AI 放手干活，又怕它擅自做危险的事（比如删文件）。/permissions 就是给 AI 立规矩：哪些事直接允许、哪些要先问你一句、哪些干脆禁止。你把这三档分好，它就照章办事，不越界也不啰嗦。就像家里请的钟点工，你提前讲好：扫地拖地随便做（允许），扔东西前先问我（询问），我书房别进（拒绝）。规矩清楚了，你放心，她也省心。想控制 AI 能干什么、不能干什么时，来这里设。

何时用 想规定 AI 哪些操作可做、哪些需确认时。

/skills

— 查看现成的技能包

烦恼解决：有些活儿步骤固定又繁琐，每次从头教 AI 太累。技能（skill）就是别人提前打包好的『一键流程』——把一整套操作封装成一个现成的本事，AI 直接调用。/skills 就是列出当前都有哪些这样的技能包，还能按谁更费资源排序、把不常用的藏起来。就像厨房里的预制菜或料理包，复杂的菜不用你从切菜开始，撕开包装下锅就行。想知道 AI 现在会哪些打包好的本事，来这里翻菜单。

何时用 想查看或整理已有的打包工作流时。

/reload-skills

— 刷新技能列表

烦恼解决：你刚往技能或命令文件夹里加了个新本事，或者改了某个，但 AI 还按老的来，没认出新的。/reload-skills 就是让它原地重新扫一遍这些文件夹，把新增和改动认出来，不用关掉重开。就像你往点歌机里加了新歌，不必拔电源重启，按个『刷新曲库』就能搜到。改完技能立刻生效，省去重启的等待。刚加了或改了技能、命令，发现 AI 没认出来时用。

何时用 新增或修改技能后想立刻让 AI 认出来时。

/plugin

— 管理功能插件

烦恼解决：Claude Code 自带的功能不够用，你想额外装些扩展来增强它。插件就是这种可装可卸的『附加功能模块』。/plugin 就是你的插件管理处：装新的、看已装的、启用或停掉某个。就像给浏览器装扩展，或给手机下 App——主体还是那个主体，但你能按需求往上加料，不想要了随时卸。想给 AI 增添或管理额外功能时，来这里操作。

何时用 想安装、启用或管理扩展插件时。

/reload-plugins

— 刷新已启用插件

烦恼解决：你改了某个插件的设置或内容，但它还在按旧的运行，改动没生效。/reload-plugins 就是让已经开着的插件重新加载一遍，把你的改动应用上，不用整个关掉重来。就像你改了手机 App 的设置，有时得退出来重进一次才生效——这个命令帮你一键重进，立刻看到新效果。改完插件、想让改动马上起作用时用。

何时用 修改插件后想让改动立即生效时。

/add-dir

— 多开一个工作目录

烦恼解决：AI 默认只能看你当前的项目文件夹，但你这次的活儿还牵扯到另一个文件夹里的东西，它够不着。/add-dir 就是临时给它多开一扇门，把另一个文件夹也纳入它本次能访问的范围。就像你在书桌上干活，突然需要旁边柜子里的资料，就把那个抽屉也拉开放手边——这样两边的东西都能拿。需要 AI 同时处理多个文件夹的内容时用，仅对本次会话有效。

何时用 本次任务要用到当前项目外的另一个文件夹时。

/sandbox

— 开关安全隔离模式

烦恼解决：让 AI 跑一段不确定安不安全的操作，你怕它影响到电脑上别的东西。沙箱模式就是把它关进一个『隔离的小房间』里运行，里面随便折腾，出不来也波及不到外面，更安全。/sandbox 就是开关这个隔离模式（仅部分平台支持）。就像让孩子在专门的游戏围栏里玩，玩具撒一地也只在围栏内，不会弄乱整个客厅。要跑有风险的操作、又想保护好其他文件时，先把它打开。

何时用 想让操作隔离运行、更安全时。



代码审查与验证 8 / 8

挑 bug、做精简、查漏洞，再真把应用跑起来确认有效。

/diff

— 看还没保存的改动

烦恼解决：AI 改了一堆代码，你想知道到底动了哪些地方、改前改后差在哪，一行行翻太费劲。/diff 打开一个对比查看器，把还没正式提交的改动并排列出来：哪行加了、哪行删了、哪行变了，一目了然。就像你交报告前用 Word 的『修订模式』看一遍，红的删、蓝的加，改了什么清清楚楚。AI 干完活、你想先验收一遍再决定要不要留时用。

何时用 想检查 AI 改了哪些还没提交的内容时。

↔ **辨析** 和 /code-review 比：diff 只展示改了哪些行，不评判、也不动手改。

/review

— 本地审一份代码请求

烦恼解决：团队里有人提交了一份代码改动请求（PR，就是『我改好了这些，求合并进项目』的申请），你得审一遍才敢放行，但来回切网页很麻烦。/review 让你直接在当前会话里把这份 PR 拉过来本地审查，AI 帮你看看改了些什么、有没有问题。就像同事递来一份方案让你签字，你不用跑去他工位，坐自己桌前摊开看完再批。需要评审别人提交的代码改动时用。

何时用 要审查别人提交的代码合并请求（PR）时。

↔ **辨析** 和 /code-review 比：review 审的是【别人提交的 PR】；code-review 审你当前这次改动。

/code-review 参数 low~ultra · --fix · --comment

— 挑代码毛病和啰嗦

烦恼解决：代码改完了，但里面可能藏着 bug，或者写得绕、能更简洁，你自己不一定看得出。/code-review 让 AI 当审稿人，既找出可能出错的地方，也指出哪里能精简复用。可以调审查力度（从轻到狠，ultra 是动用云端一队 AI 深挖）；加 --fix 让它直接帮你改好，加 --comment 把意见写成评论贴回去。就像请位资深编辑通读你的稿子，错别字和啰嗦句一起圈出来，要狠还能逐字推敲。改完代码、想让它把关一遍时用。

何时用 想给改动找 bug 并顺手优化时。

↔ **辨析** 和 /simplify 比：code-review 找 bug + 顺手优化；simplify 只优化。和 /security-review 比：后者只盯安全漏洞。

/simplify

— 只做精简优化

烦恼解决：代码能跑，但写得啰嗦、重复、绕弯子，你只想让它更干净利落，不需要查 bug。/simplify 就专做这一件事：精简多余、复用重复、提升效率，而且会直接帮你改好。注意它只管『清理优化』，不负责找错——找 bug 那是 /code-review 的活。就像请人来帮你整理房间，把堆叠的杂物归位、重复的东西扔掉、动线理顺，但不检修水电。只想把代码收拾得更利索时用。

何时用 只想让代码更简洁高效、不查 bug 时。

↔ **辨析** 和 /code-review 比：只做精简优化、不找 bug，而且会直接帮你改。

/security-review

— 专查安全漏洞

烦恼解决：代码看着没问题，但可能藏着安全隐患——比如密码没保护好、有被人钻空子的口子，普通审查不一定盯这些。/security-review 专门给你当前这条分支上的改动做安全体检，揪出可能被坏人利用的漏洞。就像房子盖好后请人专门来查防盗：门窗锁牢没、有没有谁能翻进来的缝，跟验收装修是两码事。改动涉及用户数据、登录、支付这类敏感地方，上线前先用它扫一遍。

何时用 改动上线前想排查安全漏洞时。

↔ **辨析** 和 /code-review 比：只查安全漏洞这一类，不管普通 bug 和优化。

/run

— 真把应用跑起来看

烦恼解决：代码改完了，但光看文字不放心——到底有没有真的生效？/run 帮你把项目应用真正启动起来并实际操作一下，让你眼见为实地看改动有没有起作用。就像菜谱改了配方，光看纸上写的没用，得真下锅炒一盘尝尝才知道行不行。光读代码看不出效果、想亲眼确认时用它把东西跑起来。

何时用 想真正启动应用、亲眼看改动是否生效时。

↔ **辨析** 和 /verify 比：run 只把应用跑起来给你看；verify 还会对照验收标准核对达标。

/verify

— 跑一遍确认改动有效

烦恼解决：你说『这个 bug 修好了』，但嘴上说不算数，得证明确实管用。/verify 会帮你把应用构建、运行起来，并观察实际表现，来确认这次改动真的有效、没白改。比 /run 更进一步——它不只是跑起来，而是带着『验收标准』去核对结果对不对。就像修好的水龙头，不能只拧开看看，得真接一盆水、确认不漏了才算修好。想给一次修改盖个『确实修好了』的章时用。

何时用 想确认一次修改确实达到了预期效果时。

↔ **辨析** 和 /run 比：verify = 跑起来 + 核对【确实修好了】，比 run 多一步验收。

/run-skill-generator

— 教 AI 怎么启动你的应用

烦恼解决：/run 和 /verify 想帮你把应用跑起来，但每个项目启动方式都不一样，它一开始未必知道你这个该怎么开。/run-skill-generator 就是手把手教会它『你这个项目要这样构建、这样启动』，之后它再跑 /run、/verify 就熟门熟路了。就像新来的助理第一次用你家那台老式咖啡机，你先示范一遍：先加水、按这个键、等绿灯——教会一次，往后他自己就会操作了。第一次让 AI 跑你这个应用、它不太会启动时用。

何时用 首次配置 /run、/verify 如何启动你的项目时。



并行与后台 10 / 10

先规划再动手，把大活拆开并行跑，或丢到后台与云端定时干。

/plan

— 先想清楚再动手

怕 AI 一上来就乱改、改完发现方向错了白忙活？这条命令让它先把方案讲给你听。就像请装修师傅别急着砸墙，先出一张设计图：怎么改、动哪几处、为什么这么做，全列清楚等你点头。你觉得不对就当场拦下来，省得拆错了再返工。等方案谈拢了，它才正式开工。适合改动稍大、心里没底、想先对齐思路的时候用，把返工的风险挡在动手之前。

何时用 改动较大、想先确认方案再让它动手时。

↔ **辨析** 和 /ultraplan 比：plan 在终端内轻量出方案；ultraplan 更重，走浏览器审查再执行。

/tasks /bashes

— 查看后台活儿

你让 AI 同时在后台跑了好几摊活，过会儿就忘了哪个干完了、哪个还卡着？这条命令给你拉一张总清单，像餐厅后厨的取餐叫号屏：哪几锅菜在炖、哪锅好了、哪锅出问题了，一眼看全。你还在这里管理它们——盯进度、看结果、把不要的停掉。手头并行的任务一多就靠它统筹，免得自己在脑子里记一堆账，乱成一团。

何时用 同时跑了多个后台任务、想统一查看管理时。

/background /bg

— 让它自己干去

有些活儿要跑挺久，你却得守在旁边干等，啥也干不了？这条命令把当前这摊对话整个甩到后台，让它变成一个自己埋头干活的小助手，你这边立马腾出手去聊别的、忙别的。就像把一锅汤交给慢炖锅，设好了就走人，不用一直盯着火。它在后台默默推进，你想起来了再回去看进度。适合那种耗时长、又不需要你时刻盯着的任务。

何时用 任务耗时长、想腾出手去忙别的时。

/stop

— 喊停后台会话

甩到后台的活儿，发现方向不对或者根本不需要了，想立刻叫停怎么办？

这条命令就是那个急停按钮，一按，正在后台跑的这个会话马上收手。

就像你交给跑腿的人去办事，半路改主意了打个电话喊他回来，别白跑一趟、别白花力气。注意它只在你跟这个后台会话还连着线的时候才好使——断了联系就够不着了。想及时止损时用它最干脆。

何时用 想立刻叫停一个还连着的后台会话时。

/batch

— 大活拆开同时干

一个大改动要动几十处，一处处排队改慢得让人抓狂？

这条命令把大活拆成五到三十个小块，每块派一个小助手在各自独立的工位上同时开干，互不打架，干完各自交一份「改动申请单」（PR，就是把修改提上去等人审核合并的请求）。好比一面墙要刷漆，不是一个人从头刷到尾，而是叫一队师傅一人一段同时刷，唰一下就齐活。适合改动多又能拆开、想抢时间的大工程。

何时用 大改动能拆成多块、想并行快速完成时。

/workflows

— 看活儿干到哪了

一堆任务在跑，你想知道整体进行到哪一步、能不能喘口气暂停一下？

这条命令打开一块进度总览屏，把正在走的流程摊在你眼前，

像工厂车间墙上的生产看板：每条线走到哪、整体啥节奏，看得明明白白。

你还能在这儿点暂停、点继续，或者把当前进度存下来回头再接着干。适合多任务并行、想随时掌控全局节奏的时候用，不至于两眼一抹黑。

何时用 想总览并暂停/恢复/保存正在跑的流程时。

/deep-research

— 撒网查资料写报告

想查个事，自己开十几个网页一篇篇翻、还得辨别哪个靠谱，累到头大？
这条命令一口气撒出去好多路同时上网搜、抓资料，再把各家说法摆一起对账，看谁跟谁能对上、谁在瞎说，最后揉成一份带出处的报告给你。
就像你雇了一队调查员分头跑图书馆，回来汇总成一份注明来源的卷宗，每句话都查得到根。适合做功课、查行情、写综述这类要靠谱信息的活。

何时用 需要多来源核实、整理成带出处的报告时。

/loop /proactive

— 同一句反复跑

有件事得隔一会儿就查一遍、做一遍，你总不能定个闹钟自己手动点吧？
这条命令让它把你给的那句话反复执行，还能设隔多久跑一次。
就像让保洁阿姨每隔半小时来巡一遍卫生间补补纸，不用你每回都去喊。
只要这个对话开着，它就按你定的节奏一遍遍自动干。适合盯状态、定时检查、循环处理这类重复又规律的活，把你从反复手动里解放出来。

何时用 某件事需要按固定间隔反复执行时。

/schedule /routines

— 云端定时干活

想让某件事每天固定点自动跑，哪怕你电脑关了、人不在也照干？
这条命令帮你在云端立一份定时任务清单：你能新建、改时间、列出来看看、或手动跑一次。好比给家里定个扫地机器人的日程，每天早八点自己出来扫一圈，你睡着了它也照干。区别是它跑在云上，不靠你这台机器开着。适合每天/每周固定要做的例行活，设一次就长期自动跑。

何时用 想让任务按日程在云端自动定时运行时。

/ultraplan

— 起草审完再执行

重大的活儿，光在终端里看方案不够踏实，想摊开在大屏上仔细审一遍再放心动手？这条命令走的就是这套讲究流程：先在专门的会话里起草方案，再丢到浏览器里让你舒舒服服地审查把关，确认没问题了，才放出去执行——可以远程跑，也可以在你本机跑。就像盖楼前先出全套图纸、过完评审会才破土动工。适合分量重、必须谋定而后动的复杂任务。

何时用 重大任务想先草拟、浏览器里审查再执行时。

↔ **辨析** 和 /plan 比：ultraplan = 起草→浏览器审查→远程/本地执行，适合要正式评审的大任务。



远程与跨设备 11 / 11

在网页、手机、桌面、Slack、浏览器之间接力使用同一份工作。

/remote-control /rc

— 网页端遥控会话

人离开了电脑，但手头那个会话还想接着指挥，怎么办？这条命令给它开个远程遥控权限，之后你打开 claude.ai 网页（就是 Claude 的官方网站）就能隔空操控这个会话，跟用遥控器关电视一个意思——人不在跟前也使唤得动。好比给家里装了智能开关，出门在外用手机就能开关灯。适合需要离开座位、又想随时回来继续掌控这个会话的场景。

何时用 想离开电脑后从网页端继续操控会话时。

↔ **辨析** 和 /teleport 比：方向相反——这是让网页来遥控你这个终端会话。

`/teleport` `/tp`

— 网页会话拉回终端

之前在网页上跟 Claude 聊出来的进度，现在想搬到电脑的命令行窗口里接着干，总不能从头再来一遍吧？这条命令一下把那个网页会话整个「瞬移」到你眼前的终端里，聊到哪接着哪，无缝衔接。就像看视频换了个屏，从手机点一下就接着电视播，不用倒回开头。注意这功能需要订阅才能用。适合换设备、想把网页那头的活儿接到本地终端继续推进的时候。

何时用 想把网页端会话接到本地终端继续时（需订阅）。

↔ **辨析** 和 `/remote-control` 比：方向相反——这是把网页会话【拉回】本地终端。

`/desktop` `/app`

— 搬到桌面应用

想把当前会话挪到电脑上那个专门的 Claude Code 桌面应用里继续，界面更顺手？这条命令一键搬家，会话原样接着用。好比 you 正用手机看的剧，回家点一下就转到客厅大电视上接着追，内容不变、屏幕更大更舒服。需要你的电脑是 macOS 或 Windows 系统，并且开通了订阅才能用。适合想从命令行窗口换到更完整、更好操作的桌面应用界面里干活的时候。

何时用 想把会话转到桌面应用里继续时（需订阅）。

↔ **辨析** 和 `/teleport` 比：desktop 是把会话搬进桌面 App，不是终端与网页互拉。

/mobile `/ios /android`

— 扫码下手机版

想在手机上也能用 Claude Code，但不知道去哪下、怎么装？
这条命令直接给你弹一个二维码，掏出手机扫一扫就跳到下载页，
省得自己在应用商店里翻来翻去找半天、还怕下错。就像餐厅桌上贴的点餐码，
对着一扫菜单就出来了，干脆利落。安卓、苹果都照顾到了。适合你想让 Claude Code
装进口袋、出门在外也能随手用的时候，一扫即得。

何时用 想在手机上安装 Claude Code App 时。

/autofix-pr

— 自动救你的提交

代码改动提上去（PR，等人审核合并的修改请求），结果自动检测没过、
或者审核的人留了一堆意见，你还得一条条手动改，烦不烦？
这条命令派个网页小助手专门盯着你这次的提交，一旦自动检测（CI，
专门帮你跑一遍检查代码有没有毛病的流水线）报红、或者有人留了评论，
它就自动改好再推上去。就像交作业有个隐形助教随时帮你订正错题，不用你守着。
适合不想被反复打回、想让提交自己跑通的场景。

何时用 想让 PR 的检查失败或审查意见被自动修复时。

/remote-env

— 设网页跑活环境

你在网页上让 Claude 干活时，它默认在什么样的环境里跑、配了哪些家伙事，
总不能每次都重新交代一遍吧？这条命令让你一次性把这个默认环境配好，
以后网页会话就照这套设定开干。好比提前把厨房的锅碗瓢盆、调料都备齐摆好，
厨师一进来就能直接下锅，不用每回从头找工具。适合常用网页版、
想让每次开工的底子都统一省事的时候，设一次长期受用。

何时用 想给网页会话定好默认的运行环境时。

/web-setup

— 网页接上代码仓库

想用网页版 Claude Code 管你 GitHub 上的代码（GitHub 就是程序员存放和协作代码的网站），但俩账号还没打通？这条命令借助你电脑里现成的 gh 工具，帮你把 GitHub 账号跟网页版连起来，一次牵好线。就像给两个 App 做账号互联授权，连上之后网页那头就能直接读你的代码仓库了，不用来回手动倒腾。适合第一次配置、想让网页版能管到你 GitHub 项目的时候用。

何时用 首次把 GitHub 账号连到网页版 Claude Code 时。

/install-github-app

— 给仓库装自动助手

想让 Claude 常驻在你的代码仓库里，自动帮你干活、响应改动？这条命令给指定仓库装上 Claude 的 GitHub 助手（GitHub 是程序员存代码、搞协作的网站）。装好之后，它就像给你的仓库请了个长期值班员，能自动跟进、按规则干活，不用你每回手动喊它过来。好比给小区门口装个自动门禁系统，装一次就一直管用。适合想让某个项目长期享有 Claude 自动化服务的时候，一次安装长期生效。

何时用 想给某个 GitHub 仓库装上 Claude 自动化助手时。

/install-slack-app

— 装进团队聊天群

想在团队日常聊天的 Slack 里直接喊 Claude 干活，不用切来切去？这条命令帮你把 Claude 的 Slack 应用装进去（Slack 是很多公司用的团队聊天办公软件，类似企业版微信群）。装好后，Claude 就像被拉进了你们的工作群，在聊天框里 @ 它就能使唤，沟通和干活在一个地方搞定。好比把外卖小哥拉进小区业主群，下楼取餐一句话的事。适合团队用 Slack 协作、想让 Claude 随叫随到的时候。

何时用 想把 Claude 接入团队 Slack 协作时。

/chrome

— 调 Chrome 里的设置

想让 Claude 在你的 Chrome 浏览器里按你的偏好来干活，得先把相关设置调顺？这条命令就是那个设置入口，让你配好 Claude 在 Chrome 里的各项选项。好比新买的扫地机器人，先在 App 里圈好哪扫哪不扫、几点开工，调顺了用起来才趁手。Chrome 就是大家最常用的那个谷歌浏览器。适合你打算让 Claude 跟浏览器配合干活、想先把它在浏览器里的脾气秉性调到位的时候用。

何时用 想配置 Claude 在 Chrome 浏览器里的设置时。

/ide

— 管编辑器联动

想让 Claude 跟你写代码用的那个编辑器（IDE，就是程序员敲代码的专用软件，类似写文章用的 Word）打通配合，又不确定有没有连上？这条命令帮你管理这种联动，还能直接告诉你当前接没接上、状态如何。好比检查手机和蓝牙音箱配没配对成功——连上了放歌才出声。适合你想让 Claude 跟编辑器协同干活、或者怀疑没连好想确认一下状态的时候，一看便知，省得瞎猜。

何时用 想管理或查看编辑器（IDE）集成状态时。



界面与配置 10 / 10

主题、配色、状态栏、快捷键、全屏——把工具调成你的顺手样。

/config /settings

— 总设置面板

想统一调整这个工具的方方面面？这条命令打开“总控制台”。就像手机的设置 App，所有能改的都在这一个地方：换配色主题、挑默认模型、调输出风格……不用记一堆零散命令，进来一个面板全搞定。看不顺眼、用不顺手的地方，都能在这里改到合心意。它是你个性化整个工具的大本营，所有偏好设置的入口。

何时用 想集中调整主题、模型、风格等各项设置时。

/status

— 查体检报告

不确定工具现在啥状态、是不是连上了？这条命令帮你查“体检报告”。好比开车前瞄一眼仪表盘：油够不够、引擎正不正常、一目了然。它会告诉你现在用的是哪个版本、哪个 AI 模型、登录的是哪个账号、网络连接顺不顺。出问题时先来这看一眼，心里就有底了，排查毛病也快。一句话：你和工具当前所有关键信息的“现状一览”。

何时用 想确认版本、账号、模型或连接是否正常时。

/theme

— 换配色皮肤

盯着屏幕眼睛累，或单纯想换个心情？这条命令帮你换“皮肤”。就像给房间换个灯光氛围：白天来个亮色清爽，晚上切深色护眼，还能跟着系统自动切换。它专门照顾各种需要——有给色弱朋友优化的配色，有复古风的，也能自己捏一套喜欢的。看着舒服了，用起来才有心情。纯粹是让界面顺你的眼，跟功能强不强没关系。

何时用 看腻了当前配色，或想护眼、想换风格时。

/color 参数 颜色 / default

— 标记输入框颜色

同时开好几个窗口，经常搞混在哪个里打字？这条命令帮你给当前这个输入框“贴个色标”。就像给一摞文件夹贴不同颜色的标签，扫一眼就知道哪个是哪个。给这次对话的打字栏设个专属颜色，多窗口并排时一眼就能认出“哦，这个是它”，不会手忙脚乱发错地方。填具体颜色就上色，填 default 就恢复原样。小功能，但开多窗口的人会很爱。

何时用 同时开多个窗口、想一眼区分时。

/statusline

— 定制底部信息条

屏幕底部那条小信息栏，想让它显示你最在意的东西？这条命令帮你定制它。就像汽车仪表盘可以自己选要盯哪几个表：有人爱看时速，有人爱看油耗。这条状态栏也一样，你能安排它显示对你最有用的信息，不用被默认的将就。常用的关键信息摆在眼皮底下，随时一瞥就知道，不用专门去查。让那条最不起眼的小条，变成最顺手的助手。

何时用 想让底部信息栏显示你最关心的内容时。

/keybindings

— 自定义快捷键

总觉得某个操作按起来别扭、不顺手？这条命令让你自己改快捷键。就像把厨房里最常用的勺子挪到伸手就够的抽屉，顺手了干活才快。它帮你打开快捷键配置，或者新建一套，把各种操作绑到你习惯的按键上。用熟之后肌肉记忆一上来，手指比脑子还快。这是把工具调教成“你的专属顺手工具”的关键一步，越用越贴手。

何时用 嫌默认快捷键别扭，想改成自己习惯时。

/terminal-setup

— 修好回车换行

想在打字时换行，结果一按回车就把话发出去了？这条命令专治这个烦恼。它帮你的命令行窗口配好按键，比如让 Shift+回车变成“换行不发送”，写多行内容就顺畅了。好比新搬进办公室先把打印机、网络都接好，后面用起来才不卡壳。它只在确实需要设置的窗口里才冒出来，不需要时自动隐身，不打扰你。一次配好，长长久久舒服。

何时用 想换行却老是误发、需要配好终端按键时。

/tui 参数 default/fullscreen

— 切窗口显示方式

嫌界面挤、想让工具铺满整个屏幕沉浸用？这条命令帮你切换“显示模式”。就像看视频时点那个全屏按钮：default 是常规小窗，跟其他东西挤在一起；fullscreen 是全屏铺开，整个画面都归它，看着更开阔、更专注。内容多、想沉下心干活时切全屏特别爽，什么都看得清清楚楚。想回到平时的样子，再切回 default 就行。一键决定它“占多大地方”。

何时用 想让界面铺满屏幕、沉浸式使用时。

/focus

— 进专注模式

东西太多看花眼、想把注意力收回来？这条命令帮你切到“专注视图”，把杂七杂八收起来，只留你正盯着的重点。好比看书时拿张纸盖住别的行，眼睛只跟着当前这一行走，不容易分神。要提醒的是：它只在全屏模式下才能用，得先切到全屏。适合需要静下心啃硬骨头的时候，屏幕清爽了，脑子也跟着清爽。一键给眼睛和注意力“减负”。

何时用 全屏下东西太多、想收起干扰专心时。

/scroll-speed

— 调滚轮快慢

滚鼠标滚轮翻内容，嫌一下窜太远、或太磨蹭？这条命令帮你调滚动的快慢。就像调电风扇档位：嫌慢就加快，几下翻到底；嫌晃眼就放慢，一行行稳稳看。调到手感最舒服的那一档，翻长内容时既不会刷过头、也不用磨半天。注意它只在全屏模式下能用，得先切全屏。看着是小细节，可天天滚动，顺手了体验差很多。

何时用 全屏下嫌滚轮翻得太快或太慢时。



账户与用量 7 / 7

登录登出、查账单额度、升级套餐、隐私开关。

/login

— 刷卡进门

想用工具，第一步当然是先证明你是你。这命令就像进公司刷工卡：闸机一开，系统才知道是哪位员工、能用哪些权限。/login 让你登录自己的 Anthropic 账号，登录之后工具才知道你买的是哪个套餐、能用多少额度，所有花费也记在你名下。第一次用、或者换了台电脑、或者之前退出了登录，都得先刷这道门。刷完就能正式开干，不用再反复证明身份。

何时用 第一次使用或换设备时，先登录账号。

/logout

— 刷卡出门

用完想干净退出，就用这个。就像下班离开公司刷卡出门，把工卡收回——别人再坐到这台电脑前，也进不了你的账号、花不了你的钱。如果你用的是公用电脑，或者要把电脑借给别人、转手卖掉，退出登录就是把自己那扇门锁上，账号和额度都安全。退出后下次想用，再 /login 重新刷卡进门即可，资料都还在。

何时用 用公用电脑或要离开时，退出保护账号。

/usage /cost /stats

— 查这月账单

总担心钱不知不觉花超了？这命令就是随时打开手机看话费账单：这次用了多少、套餐里还剩多少额度、最近活跃情况一目了然。心里有数才不会月底被吓一跳。如果你是付费套餐，账单还分得更细，哪块花得多看得清清楚楚，方便你掂量着用。不用记在脑子里，想看随时调出来，花钱透明、用着踏实。

何时用 想知道花了多少、还剩多少额度时。

↔ **辨析** 和 /usage-credits 比：usage 只是【查】花费和额度，不涉及续费。

/usage-credits 曾叫 /extra-usage

— 话费充值续杯

最怕的就是干到一半额度用光、活生生被卡住。这命令就像手机话费的自动充值：提前设好，套餐额度用完了也不断线，能接着往下用，该交的钱另算。相当于给自己留个后路，关键时刻不掉链子。设好之后就不用一直盯着额度发愁，工具自己续杯，你专心干活就行。适合用量大、不想被中途打断的人。

何时用 怕额度用光中断，想超额后继续用时。

↔ **辨析** 和 /upgrade 比：这是当前套餐超额后续用（另计费）；upgrade 是换更大套餐。

/upgrade

— 升级更大套餐

现在的套餐不够用、老觉得额度紧巴巴？这命令直接帮你打开升级页面，就像手机流量老超，干脆换个更大的套餐包，省得每月精打细算。点一下就跳到升级界面，按指引换更高档位，额度更宽裕、用着更舒坦。注意这是给已经在付费的 Pro/Max 用户准备的——已经在花钱了，往上再加一档而已。

何时用 现有套餐额度不够用、想换更高档时。

↔ **辨析** 和 /usage-credits 比：upgrade 换更高档套餐；usage-credits 只是临时超额续杯。

/passes

— 送朋友体验券

觉得这工具好用，想拉朋友一起玩？这命令能让你分享一周的免费 Claude Code 给朋友，就像办了张健身卡送你一张周卡体验券，朋友不用先掏钱就能试用一礼拜。好东西自己用着爽，分享出去更带劲。不过这功能得你符合条件才会出现，没看到就说明暂时还没轮到你，看到了就大方送人吧。

何时用 想免费请朋友体验一周时（符合条件才有）。

/privacy-settings

— 管隐私开关

在意自己的数据怎么被使用？这命令就是工具的隐私设置面板，像手机里那个权限管理页：你的对话和数据会不会被拿去用、用到哪一步，都在这儿看清楚、自己说了算。想关的开关一目了然，掌控感拉满。这是给 Pro/Max 用户开放的，在意隐私的人可以进去翻一翻，按自己的心意调，不放心的地方直接关掉。

何时用 想查看或调整数据隐私设置时（Pro/Max）。



出毛病时体检排查、看更新、减少弹窗、查可用命令。

/doctor

— 工具体检

工具时灵时不灵、不知道哪儿出了毛病？这命令就是给它做个全身体检：自动检查安装对不对、设置有没有问题，哪里不舒服一查便知。更省心的是，查出问题后按一下 f 键，它能自动帮你修好，跟医生当场开药顺手治了一样，不用你自己瞎折腾。工具感觉怪怪的、装完跑不起来，先来这儿体检一遍，多半能找出病根。

何时用 工具出毛病、装完跑不起来时，先体检自查。

/debug

— 开行车记录仪

遇到怪问题但说不清哪儿错了？这命令就像给车装上行车记录仪：打开后，工具会把背后每一步都详细记下来，出了状况翻记录就能看清到底哪个环节卡了壳。平时不用开，一旦碰上反复出错、又找不出原因的疑难杂症，开着它再复现一遍问题，日志里往往就藏着真凶。给技术支持反馈时，这份记录也是最有力的证据。

何时用 碰上反复出错又查不出原因时，开日志取证。

/feedback /bug /share

— 意见箱投信

用着不顺、发现 bug、或者想夸两句，憋着没处说？这命令就是工具的意见箱：报告毛病、提建议、甚至把一段对话分享出去，都从这儿一键投递给开发团队。就像餐厅桌上那张顾客反馈卡，你的吐槽和点子真能传到做饭的人手里。遇到 bug 别自己干生气，投个信说不定下个版本就修了，你的反馈实打实地让工具变好。

何时用 想报 bug、提建议或分享对话时。

/heapdump

— 拍内存快照

工具越用越卡、像是在偷偷吃内存？这命令能给它的内存现状拍张详细快照，连同占用明细一起存到你桌面上，就像怀疑身体哪儿堵了去拍个 CT 片子，把内部情况照得清清楚楚。普通人平时用不上，主要是工具内存异常、卡顿严重时，把这张片子交给技术人员，他们对着明细就能找出是谁在偷吃内存。出问题时的诊断利器。

何时用 工具异常卡顿、疑似内存问题时，留诊断快照。

/release-notes

— 看更新日志

工具悄悄更新了，但不知道这版多了啥新花样？这命令打开一个版本选择器，让你挑着看每个版本的更新日志，就像手机 App 更新后那句「本次更新内容」，告诉你新增了什么、修好了哪些毛病。好奇心强、想跟上新功能的人，进来翻翻就知道最近又添了哪些好用的东西，不至于守着老用法，白白错过省事的新招。

何时用 想了解新版本加了什么功能时。

/fewer-permission-prompts

— 少弹确认框

每点一下都要你确认「允不允许」，烦得要死？这命令帮你翻一遍历史记录，把那些纯粹看看、不会改坏东西的安全操作（比如只读不写的）记进白名单，以后这类小事就不再反复弹框问你了。就像常来的熟客进门保安不再每次拦着查证件——放行名单上的，直接进。该拦的危险操作照样拦，只是把没风险的唠叨给省了，用着清爽不少。

何时用 嫌权限确认框弹太多、想少被打扰时。

/help

— 翻说明书

命令太多记不住、不知道还能干啥？这命令就是工具自带的说明书：一打开，所有能用的命令和帮助都列出来，哪个干啥用一目了然。

就像买了新家电先翻随机附带的使用手册，不用到处瞎猜。新手刚上手、或者老用户突然忘了某个命令叫啥，进来扫一眼准能找到。迷路时的指南针，记不清的时候第一个该想到它。

何时用 忘了有哪些命令、不知道怎么用时。



体验与杂项 7 / 7

语音、电台、教学课、使用报告、贴纸、API 迁移等额外彩蛋。

/voice 参数 hold/tap/off

— 动嘴不动手

手忙脚乱顾不上打字，或者就是懒得敲键盘？这命令打开语音听写，让你直接对着说，它帮你把话转成文字，就像微信里那个按住说话的语音转文字，动动嘴就行。

还能调成长按说话、点一下说话、或干脆关掉三种模式，随你顺手。打字累、想边走边说的时候特别香。注意它得配合 claude.ai 账号用，没账号的话先把这步备齐。

何时用 懒得打字、想动嘴说话输入时。

/radio

— 工作背景音乐

干活太安静、心里发慌，又懒得自己找歌？这命令一键打开 Claude FM 轻音乐电台，给你的工作配上不吵不闹的背景音，就像咖啡馆里那种恰到好处的轻音乐，让人静下心来。不用切出去开音乐软件、不用挑歌单，点一下就有，纯粹陪你专注干活。需要点氛围又不想被打扰的时候，开着它，效率和心情一起上来。

何时用 想边干活边听点轻音乐放松时。

/powerup

— 带演示的教学课

光看文字说明记不住、想有人手把手教？这命令开一堂带动画演示的互动教学课，边演边教，把功能怎么用活生生地展示给你看，就像新员工入职时那种边操作边讲解的实训课，比干巴巴啃手册好懂太多。新手想快速摸清门道、或者老用户想挖挖没用过的本事，上这节课最直观。看一遍动手跟一遍，功能就真正变成自己的了。

何时用 想通过动画演示快速学会用法时。

/insights

— 我的使用报告

好奇自己到底是怎么用这工具的、有没有用在刀刃上？这命令生成一份专属使用分析报告，把你的使用习惯、用得多用得少的地方都摆出来，就像支付宝年底那份「你今年这样花钱」的账单回顾，看完对自己的习惯一目了然。想复盘、想用得更聪明的人，看看这份报告就知道哪儿能改进、哪些好功能你还没碰过。了解自己，才能用得更顺手。

何时用 想复盘自己的使用习惯和情况时。

/team-onboarding

— 团队上手指南

团队里新人来了，不知道这工具该怎么教、从哪讲起？这命令根据你们实际的使用历史，自动生成一份团队上手指南，就像老员工把自己摸索出的经验整理成一份「新人入职手册」，照着学就能快速跟上，不用每个新人都从头踩坑。带团队、要拉同事一起用的人特别省事，一份指南发下去，大家上手节奏统一，少走很多弯路。

何时用 要带团队或新同事快速上手时。

/stickers

— 领工具贴纸

喜欢这工具、想搞点周边贴在电脑上炫一炫？这命令直接帮你订购 Claude Code 的实体贴纸，就像追星买周边、给爱用的 App 添点仪式感，贴本子贴电脑都好看。纯属好玩和情怀，跟干活没关系，但用得顺手又有感情的话，弄一套贴纸装点一下也挺有意思。喜欢这个工具的人，顺手薅一套，给桌面添点个性。

何时用 喜欢这工具、想订一套周边贴纸时。

/claude-api

参数 migrate / managed-agents-onboard

— 升级旧代码助手

这个偏给会写代码的人用：它帮你加载 Claude 接口的官方参考资料，最实用的是当新版模型出来、你旧代码需要跟着改时，它能帮你把代码从老版本平稳搬到新版本。打个比方，就像换了新款手机充电口，得有个转接头帮你把老设备接上去用，不至于全部推倒重来。完全不写代码的人用不上这条，知道它是给开发者搬家用的就行。

何时用 开发者想迁移代码到新模型版本时。



云服务商配置 2 / 2

接亚马逊 Bedrock 或谷歌 Vertex 云时的认证与区域设置。

/setup-bedrock

— 接亚马逊云

这条是给走亚马逊云（Bedrock）这条路的用户配置用的：帮你把账号认证、用哪个地区、用哪个模型这些设置一次配好，就像新买的家电得先接通电源、调好频道才能正常看。配置对了，工具才能顺着亚马逊云这条线跑起来。注意它只在你启用了 Bedrock 时才冒出来，没用这条路的人根本看不到，看到了再按指引一步步填即可。

何时用 用亚马逊 Bedrock 时，配置认证和区域。

/setup-vertex

— 接谷歌云

这条是给走谷歌云（Vertex AI）这条路的用户配置用的：帮你把账号认证、用哪个项目、用哪个地区一次设好，就像入住新房得先开通水电、登记好门牌才能正常住。配置妥当，工具才能顺着谷歌云这条线跑通。它同样只在你启用了 Vertex 时才出现，没走这条路的人压根看不到，需要的人照着提示把几项填齐就行，不用自己瞎摸索。

何时用 用谷歌 Vertex AI 时，配置认证和项目。

数据来源：Claude Code 官方命令参考

code.claude.com/docs/en/commands（截至 2026 年 6 月）。命令可见性随平台、套餐与环境而异，故此处罗列的注册表大于任一用户实际所见；已折叠别名、剔除 `/vim`、`/pr-comments` 等已下线命令。

PinPin Visual Identity

研究工作台 · 多智能体并发调研出品

© 2026 张拼拼 (Max Pin) · zpplife@gmail.com · 保留所有权利